

MathViz 3D — Rust Core (`mathviz_core`) Build Guide

Scope: This document covers the design, structure, and implementation guidance for the Rust computation core only. Integration with the Python Visualization Service (PyO3 bindings) is described at the boundary level. No other system layers are covered here.

1. Purpose and Role of the Rust Core

The Rust crate `mathviz_core` is the numerical engine of MathViz 3D. Its sole responsibility is to accept computation specifications from Python and return geometry buffers and numerical results as fast as possible — typically in under 80 ms for a full multi-step batch.

It must never:

- Make network requests
- Access the file system (except through explicit, audited unsafe blocks for SIMD)
- Hold mutable global state between calls
- Parse raw expression strings (Python pre-parses and sends ASTs)

It must always:

- Be stateless across calls (all state is passed in per-call)
 - Be parallelism-safe (Rayon thread pools, no shared mutable state)
 - Surface errors as typed Rust errors that PyO3 converts to Python exceptions
 - Return zero-copy NumPy-compatible array views wherever possible
-

2. Crate Structure

The crate is a single library crate (`lib.rs`) compiled as a `cdylib` for PyO3, plus a `bin` target for standalone benchmarking. All public surface is the PyO3 module; all internal modules are private Rust.

```

mathviz_core/
├── Cargo.toml
├── src/
│   ├── lib.rs          # PyO3 module root; registers all Python-callable functions
│   ├── error.rs        # Unified error type (thiserror)
│   ├── types.rs        # Shared data types: AST nodes, DomainSpec, GeometryBuffer, etc.
│   ├── evaluator/
│   │   ├── mod.rs      # Batch evaluator entry point
│   │   ├── ast.rs      # AST walker and scalar evaluation
│   │   └── constants.rs # Named mathematical constants (pi, e, golden_ratio, etc.)
│   ├── mesh/
│   │   ├── mod.rs      # Mesh generator entry point
│   │   ├── explicit.rs  # Explicit surface triangulation  $z=f(x,y)$ 
│   │   ├── implicit.rs  # Marching Cubes for  $F(x,y,z)=0$ 
│   │   └── normals.rs   # Analytic and finite-difference normal computation
│   ├── curve/
│   │   ├── mod.rs      # Curve tracer entry point
│   │   ├── tracer.rs    # Parametric and explicit curve evaluation
│   │   ├── arclength.rs # Arc-length parameterisation for draw transitions
│   │   └── discontinuity.rs # Cusp and discontinuity detection
│   ├── ode/
│   │   ├── mod.rs      # ODE solver entry point
│   │   ├── rk4.rs      # Fixed-step RK4 integrator
│   │   └── rk45.rs      # Adaptive Dormand-Prince RK45 integrator
│   ├── vector_field/
│   │   ├── mod.rs      # Vector field processor entry point
│   │   ├── arrows.rs    # Arrow geometry (instanced cone + cylinder)
│   │   ├── differential.rs # Curl and divergence via central differences
│   │   └── streamlines.rs # Streamline tracing via adaptive RK45
│   ├── riemann/
│   │   └── mod.rs      # Riemann sum rectangle geometry generator
│   └── linalg/
│       ├── mod.rs      # Linear algebra visualizer entry point
│       ├── transform.rs # Grid transformation geometry
│       ├── eigen.rs     # Analytic 2×2 and numeric 3×3 eigendecomposition
│       └── svd.rs       # SVD decomposition geometry layers
├── benches/
│   └── core_bench.rs    # Criterion.rs benchmarks
└── tests/
    └── accuracy.rs      # Numerical accuracy tests against reference values

```

3. Core Data Types (`types.rs`)

These types are the contract between Python (via PyO3 deserialization) and all internal Rust modules. They must be cheap to construct, clone-free where possible, and serializable via `serde`.

3.1 ASTNode

Represents a pre-parsed mathematical expression tree sent from Python (SymPy serialization). Python never sends raw strings; it sends the AST.

The AST must support:

- **Literals:** floating-point constants
- **Variables:** named symbols (x, y, z, t, and any user-defined parameter)
- **Unary ops:** negation, sqrt, abs, trig functions (sin, cos, tan, asin, acos, atan), hyperbolic functions, exp, ln, log, floor, ceil, sign
- **Binary ops:** add, sub, mul, div, pow, atan2, mod
- **N-ary ops:** sum and product (for series expressions)

Design notes:

- Use a recursive enum. Box children to control stack depth.
- Keep evaluation stack-based to avoid deep recursion for large expressions.
- Named constants (pi, e) should be resolved to `f64` at deserialization time, not at evaluation time.

3.2 DomainSpec

Describes the evaluation grid for one or more axes.

Each axis (x, y, z, t) has: `min: f64`, `max: f64`, `steps: usize`. Steps range from 64 to 2048. Only axes relevant to the concept type need to be populated.

Validation rules (enforced at the Rust boundary, not just Python):

- `min < max` for every axis present
- `steps` clamped to `[64, 2048]`
- At least one axis must be present

3.3 GeometryBuffer

The primary output type. Contains flat f32 arrays suitable for direct upload to WebGL.

Fields:

- `vertex_buffer: Vec<f32>` — interleaved XYZ, stride 3
- `normal_buffer: Vec<f32>` — interleaved XYZ normals, stride 3 (surfaces only; empty for curves)
- `index_buffer: Vec<u32>` — triangle indices (surfaces) or line-strip indices (curves)
- `uv_buffer: Vec<f32>` — optional UV coordinates, stride 2
- `layer_id: String` — echoed from the step descriptor for client identification
- `is_delta: bool` — true if this buffer contains only new geometry relative to the previous step

3.4 BatchRequest and BatchResult

`BatchRequest` is the top-level input to the batch evaluator. It carries:

- A list of `(hash_key: String, ast: ASTNode, domain: DomainSpec, concept_type: ConceptTypeHint)` entries
- A map of parameter values `{name: f64}` shared across all entries in the batch

`BatchResult` maps each `hash_key` to a `GeometryBuffer` (or an error per entry). Successful and failed entries are returned together so Python can report per-step failures without blocking the whole scene.

3.5 Error Type (`error.rs`)

Use `thiserror` to define a unified `MathvizError` enum with variants:

- `DomainViolation(String)` — $\min \geq \max$, steps out of range, etc.
- `EvalError(String)` — division by zero, NaN result, undefined symbol
- `MeshError(String)` — degenerate geometry, marching cubes failure
- `OdeError(String)` — integrator diverged, step limit exceeded
- `DeserializeError(String)` — malformed AST from Python
- `UnsupportedOperation(String)` — concept type not yet implemented

All variants implement `Into<PyErr>` via a `PyO3` impl block in `lib.rs`.

4. Module Design

4.1 Batch Evaluator (`evaluator/`)

Purpose: Accept a `BatchRequest`, evaluate all (AST, DomainSpec) pairs in parallel, and return a `BatchResult`.

Design:

The entry point function takes a `BatchRequest` and returns `BatchResult`. Internally it:

1. Deduplicates entries by `hash_key` — if two steps reference the same (AST + domain), compute once.
2. Distributes unique entries across a Rayon `par_iter`. Each worker owns its entry and evaluates independently with no shared mutable state.
3. Each worker calls the AST walker with a pre-allocated output `ndarray`. The walker iterates over the domain grid in row-major order. LLVM will auto-vectorize tight inner loops if the walker is written without branches on the hot path.
4. Results are collected into a `HashMap<String, Result<GeometryBuffer, MathvizError>>` using Rayon's `collect`.

AST Walker (`ast.rs`):

The walker is a recursive function that takes an `ASTNode` reference and a slice of variable bindings `&[(name, f64)]` and returns `f64`. For grid evaluation, the outer loop over grid points calls the walker per point with the current (x, y, z, t) bound as variables.

Performance-critical design choices:

- Avoid heap allocation inside the inner loop. Variable bindings are a fixed-size array on the stack for the common case of ≤ 8 variables.
- Use a match on the AST enum with `#[inline(always)]` on the recursive call. The compiler can often eliminate the recursive overhead for shallow trees.
- Special-case the most common sub-expressions: single-variable functions like `sin(x)` should reduce to a direct `x.sin()` call without a general recursive walk.

Constants (`constants.rs`):

A static lookup table mapping constant names (pi, e, tau, golden_ratio, euler_mascheroni, ln2, ln10, sqrt2) to their `f64` values. Resolved at AST deserialization time.

4.2 Mesh Generator (`mesh/`)

Purpose: Convert a scalar field (output of the batch evaluator) into a renderable triangle mesh.

4.2.1 Explicit Surface (`explicit.rs`)

For surfaces of the form $z = f(x, y)$.

Algorithm:

1. The scalar field is already evaluated as a 2D grid of z-values (row-major, shape $[nx \times ny]$).

2. Triangulate the grid: each grid quad becomes two triangles. Connectivity is fully regular, so the index buffer can be generated analytically without inspecting vertex data.
3. Remove degenerate triangles: any triangle where two or more vertices share the same position (NaN or infinite z-values produce these) is excluded from the index buffer. Do not remove vertices; only skip their indices.
4. Clamp z-values that are finite but extreme (beyond $\pm 1e6$) to a configurable render bound to prevent GPU precision issues.

Output: `vertex_buffer` (xyz for each grid point), `index_buffer` (triangle list), `normal_buffer` (computed by `normals.rs`).

4.2.2 Implicit Surface (`implicit.rs`)

For surfaces of the form $F(x, y, z) = 0$.

Algorithm: Marching Cubes on a uniform voxel grid.

Implementation steps:

1. Evaluate F over a 3D grid (up to 128^3 voxels for performance; ultra mode goes to 256^3).
2. For each voxel, classify the 8 corners as inside ($F < 0$) or outside ($F \geq 0$).
3. Look up the triangulation pattern from the standard 256-entry Marching Cubes table.
4. Interpolate edge intersection positions linearly between corner values.
5. Collect all triangles; deduplicate shared vertices using a spatial hash map.

For adaptive resolution: start at 64^3 , identify voxels where $|\nabla F|$ is large (high curvature), and subdivide those voxels to 128^3 in a second pass. This concentrates resolution where the surface bends without paying the full 128^3 cost uniformly.

Run each z-slice in parallel via Rayon, aggregating results with a channel.

4.2.3 Normal Computation (`normals.rs`)

Two strategies:

Analytic normals (preferred): If the scalar field has analytic gradient information (Jacobian passed alongside the field), compute normals directly from the gradient. The gradient at a surface point (x, y, z) is the normal direction for implicit surfaces. For explicit surfaces $z=f(x,y)$, the normal is $(-\partial f/\partial x, -\partial f/\partial y, 1)$, normalized.

Finite-difference normals (fallback): For each vertex, sample the scalar field at $\pm h$ in each axis and compute a central-difference gradient. Use $h = \text{grid spacing} \times 0.5$. Normalize the result.

In both cases, smooth normals across shared vertices by averaging the normals of all triangles that share the vertex.

4.3 Curve Tracer (`curve/`)

Purpose: Evaluate parametric or explicit curves and produce line-strip geometry.

4.3.1 Curve Tracer (`tracer.rs`)

For parametric curves $(x(t), y(t), z(t))$ or explicit 2D curves $y=f(x)$:

1. Evaluate at `steps` uniformly spaced parameter values.
2. Produce a flat `vertex_buffer` (xyz per point) and a `line_strip_index_buffer` (sequential indices with breaks for discontinuities).

For 2D curves: $z = 0.0$ for all points.

4.3.2 Arc-Length Parameterization (`arclength.rs`)

Required for the `draw` step transition animation, which reveals the curve segment-by-segment along its arc length.

1. Compute the cumulative arc length at each sample point using the trapezoidal rule over the Euclidean distances between consecutive points.
2. Normalize to $[0, 1]$.
3. Return the arc-length array alongside the vertex buffer. The client uses this to animate drawing by selecting the sub-buffer $[0 .. t]$ where t is the animation progress.

4.3.3 Discontinuity Detection (`discontinuity.rs`)

Discontinuities (vertical asymptotes, jumps) cause visual artifacts if rendered as connected line segments.

Detection strategy:

- Compute the first derivative of y (or z for 3D curves) numerically between consecutive samples.
- If the magnitude of the derivative exceeds a threshold (e.g., $10\times$ the median derivative magnitude), insert a segment break at that point.
- A segment break is represented in the index buffer as a sentinel value (e.g., `u32::MAX`) using the WebGL primitive restart feature, or by splitting into multiple sub-buffers).

Cusp detection (for parametric curves):

- Compute $|dx/dt| + |dy/dt|$. Where this sum falls below a small epsilon, the curve has a cusp. Tag the vertex but do not insert a break.

4.4 ODE Solver (`ode/`)

Purpose: Solve N initial value problems (IVPs) in parallel. Used for phase portraits and gradient descent paths.

Each IVP is: $dy/dt = f(y, t)$, $y(t_0) = y_0$, where y is a vector (up to 3D for visualization purposes) and f is defined by an `ASTNode` (or a pair of AST nodes for 2D systems).

4.4.1 Fixed-Step RK4 (`rk4.rs`)

Classic 4th-order Runge-Kutta. Use when the vector field is smooth and the step size is known in advance.

Parameters: initial condition vector, step size h , number of steps, AST for f .

The inner loop evaluates the AST 4 times per step (the four RK4 stages). Design the evaluator so it can accept a `&mut [f64]` state vector and return a derivative vector without allocation.

4.4.2 Adaptive RK45 (`rk45.rs`)

Dormand-Prince method with error-controlled step size adaptation. Use for stiff systems, streamline tracing, and any context where step size is not known in advance.

Parameters: initial condition, initial step size estimate, tolerance (absolute and relative), max steps, AST for f .

Step size control: use the standard error estimate from the 4th and 5th order solutions. Double the step if error is well below tolerance; halve it if above. Clamp step size to $[h_min, h_max]$.

Terminate when: the trajectory exits the domain bounds, max steps is reached, or the step size falls below h_min (stiff region flagged as an error).

4.4.3 Parallel Dispatch

The ODE solver entry point accepts a `Vec<IVPSpec>` (one per initial condition) and distributes them across Rayon workers. Each worker is entirely independent: its own evaluator context, its own output trajectory buffer. Results are collected into a `Vec<TrajectoryBuffer>` where each `TrajectoryBuffer` is a flat `Vec<f64>` of (x, y, z) triples at each step.

4.5 Vector Field Processor (`vector_field/`)

Purpose: Evaluate a vector field (P, Q, R) over a 3D grid and produce arrow geometry, scalar fields for curl and divergence, and streamlines.

4.5.1 Arrow Geometry (`arrows.rs`)

For each grid point, produce one arrow consisting of a cylinder body and a cone head.

Rather than generating unique geometry per arrow, use **instanced rendering**: produce a single unit arrow mesh (cylinder + cone, centered at origin, pointing +z) and a per-instance transform matrix for each grid point. The instance data is: translation (x, y, z), rotation quaternion to align +z with the field direction, scale (proportional to field magnitude after normalization).

Magnitude normalization:

- Compute the 90th-percentile magnitude across the grid. Scale all arrows so the 90th-percentile arrow has a unit length. This prevents a few extreme values from making all other arrows invisible.
- Optionally apply log-scale compression for fields with very large dynamic range.

Output: unit arrow `GeometryBuffer` + flat instance buffer (translation + quaternion + scale per instance as f32 arrays).

4.5.2 Curl and Divergence (`differential.rs`)

Compute from the evaluated vector field (P, Q, R) on a regular grid using 2nd-order central differences.

Divergence: $\partial P / \partial x + \partial Q / \partial y + \partial R / \partial z$ at each interior grid point.

Curl: $(\partial R / \partial y - \partial Q / \partial z, \partial P / \partial z - \partial R / \partial x, \partial Q / \partial x - \partial P / \partial y)$ at each interior grid point.

Boundary points use one-sided differences. Return both as scalar/vector field grids for color-mapping by the frontend.

4.5.3 Streamlines (`streamlines.rs`)

Trace streamlines by integrating the vector field as an ODE. Each streamline is one IVP: $dy/dt = F(y)$ where F is the vector field.

Reuse the RK45 integrator from `ode/rk45.rs` with the vector field AST as the derivative function.

Seed points are distributed on a regular grid or a user-specified set. Terminate each streamline when it exits the domain or after a max arc-length.

4.6 Riemann Sum Generator (`riemann/`)

Purpose: Produce rectangle mesh geometry for Riemann sum visualizations, designed for step-by-step reveal.

Accepts: function values on the x-domain grid, subdivision count n, partition method (left, right, midpoint, trapezoid), and an optional `from_index` for incremental generation.

For each subinterval $[x_i, x_{i+1}]$:

- Compute the representative x-value based on the partition method.

- Look up (interpolate if needed) the function value at that x .
- Generate a flat rectangle mesh: 4 vertices $(x_i, 0, 0)$, $(x_{i+1}, 0, 0)$, $(x_{i+1}, f, 0)$, $(x_i, f, 0)$; 2 triangles; normal = $(0, 0, 1)$.

When `from_index` is provided, only generate rectangles from index `from_index` onward. This enables the Python Step Sequencer to request only the new rectangles added at each step without recomputing the full set.

Degenerate case: if $f(x) = 0$ for a subinterval, generate a zero-height rectangle (still valid geometry, just flat).

4.7 Linear Algebra Visualizer (`linalg/`)

Purpose: Produce geometry for matrix transformation, eigenvector, and SVD visualizations.

4.7.1 Grid Transform (`transform.rs`)

Inputs: a 2×2 or 3×3 matrix M , and a grid domain.

Steps:

1. Generate a regular grid of line segments (a lattice) in the input space. This is the "before" layer.
2. Apply M to every vertex: $v' = M \cdot v$. This is the "after" layer.
3. Return both layers as separate `GeometryBuffer` objects. The `transform` step transition in the frontend will morph between them via vertex interpolation.

Grid density: configurable via render hints; default 10×10 cells for 2D, $5 \times 5 \times 5$ for 3D.

4.7.2 Eigendecomposition (`eigen.rs`)

For 2×2 matrices: closed-form analytic solution using the characteristic polynomial. Returns two eigenvalues (real or complex) and corresponding eigenvectors.

For 3×3 matrices: QR iteration algorithm. Perform up to 1000 iterations or until the off-diagonal elements fall below $1e-10$. Return three eigenvalues (may be complex) and eigenvectors.

Eigenvalue geometry: for each real eigenvector, generate a double-headed arrow scaled by the eigenvalue magnitude. Color-code by sign (positive = one color, negative = another, configured via render hints).

If eigenvalues are complex, generate a rotation-scale indicator (an ellipse showing the spiral behavior) rather than an arrow.

4.7.3 SVD Decomposition (`svd.rs`)

Decompose $M = U \cdot \Sigma \cdot V^T$ for 2×2 and 3×3 matrices.

Use a Jacobi SVD algorithm for small matrices (exact for 2×2 , iterative for 3×3).

Produce three geometry layers corresponding to the three transformation stages:

- Layer 1: V rotation — shows the input grid rotated by V^T .
- Layer 2: Σ scaling — shows the rotated grid scaled by the singular values.
- Layer 3: U rotation — shows the final result rotated by U .

Each layer is a complete transformed grid, not a delta, so the frontend can show each stage independently.

5. PyO3 Boundary (`lib.rs`)

This section defines what Python sees. Every public function is decorated with `#[pyfunction]` and registered in the `#[pymodule]` macro.

5.1 Public Functions

All functions accept Python objects and return Python objects. Use `pyo3-numpy` to return results as `PyArray` objects for zero-copy NumPy sharing.

`batch_evaluate(py, request_json: &str) -> PyResult<PyObject>`

- Deserializes `BatchRequest` from JSON.
- Calls the batch evaluator.
- Returns a Python dict mapping hash keys to `GeometryBuffer` dicts.

`trace_curve(py, ast_json: &str, domain_json: &str) -> PyResult<PyObject>`

- Traces a single curve.
- Returns vertex buffer, index buffer, and arc-length array as NumPy arrays.

`solve_ode_batch(py, ivps_json: &str) -> PyResult<PyObject>`

- Solves a list of IVPs.
- Returns a list of trajectory arrays.

`process_vector_field(py, request_json: &str) -> PyResult<PyObject>`

- Evaluates the field, generates arrows, computes curl/divergence.
- Returns arrow instance data and scalar field grids.

`generate_riemann(py, request_json: &str) -> PyResult<PyObject>`

- Generates rectangle geometry.
- Returns vertex and index buffers for new rectangles only if `from_index` is set.

`visualize_linear_transform(py, matrix_json: &str, domain_json: &str) -> PyResult<PyObject>`

- Computes grid transform, eigenvectors, and SVD layers.
- Returns separate geometry buffers for each layer.

5.2 Error Propagation

Every public function wraps its internal call in a `match` on `Result<_, MathvizError>`. `MathvizError` implements `Into<PyErr>` mapping each variant to an appropriate Python exception type (e.g., `ValueError` for domain violations, `RuntimeError` for computation failures). Python never receives a panic — the Rust code must not panic on bad input.

Set `#[pyo3(text_signature = "...")]` on each function for Python introspection compatibility.

5.3 Thread Safety

PyO3 requires careful handling of the GIL for parallel work. The pattern to use: acquire the GIL at the boundary, release it with `py.allow_threads(|| { ... })` wrapping the Rayon work, then re-acquire to construct the return value. This allows the Rayon thread pool to run without holding the GIL, enabling true parallelism even when called from Python.

6. Performance Design

6.1 Rayon Thread Pool

Use a single global Rayon thread pool (via `rayon::ThreadPoolBuilder`) configured at module initialization. Default thread count: number of logical CPUs. Allow Python to override this via a `configure(num_threads: usize)` init function called once at service startup.

Do not use `rayon::spawn` for fire-and-forget tasks — all parallelism is scoped to a single function call and returns before the function returns. This keeps the PyO3 boundary clean.

6.2 Memory Allocation Strategy

- Pre-allocate all output buffers before the parallel section using the known grid dimensions. This avoids allocator contention during Rayon parallel execution.

- Reuse thread-local scratch buffers for intermediate computation using `thread_local!` storage. This eliminates repeated allocation for the ODE solver's stage vectors and the mesh generator's normal accumulation.
- Use `Vec::with_capacity` everywhere a buffer's final size is known analytically from the domain spec.

6.3 SIMD Considerations

- Write inner loops as simple sequential operations on `f64` slices. LLVM with `-C target-cpu=native` will auto-vectorize these.
- Avoid conditional branches inside the hot evaluation loop. Instead, evaluate the full grid unconditionally and post-process results (e.g., clamp NaN/Inf) in a separate pass.
- For complex domain coloring (1024×1024), the inner loop processes complex arithmetic per pixel — ensure the complex number representation is two adjacent `f64` values so LLVM can vectorize.

6.4 Deduplication Cache

Within a single `batch_evaluate` call, use a `HashMap<String, Arc<GeometryBuffer>>` (hashbrown for speed) to deduplicate repeated expressions. The hash key is the `hash_key` field from `BatchRequest`, computed by Python from the (AST + domain) pair. This is a within-call cache; no cross-call caching happens in Rust (that is Redis's job at the Python layer).

7. Testing Strategy

7.1 Unit Tests (`tests/accuracy.rs`)

For each computation module, test against known reference values computed by `mpmath` (high-precision Python library) and stored as fixtures.

Key test cases:

- Batch evaluator: sin, cos, exp, ln, polynomial, composite functions — check output matches reference to within 1e-9.
- Mesh generator: $z = x^2 + y^2$ (paraboloid) — check vertex count, normal directions, and that all normals point outward.
- Curve tracer: unit circle parametrically — check that the arc-length array is monotonically increasing and ends at $2\pi \times \text{radius}$.
- ODE solver: linear ODE with analytic solution — check trajectory stays within 1e-6 of the analytic path.
- Riemann sum: compare sum of rectangle areas to known integral value for $\sin(x)$ on $[0, \pi]$.
- Eigendecomposition: known 2×2 matrix with eigenvalues ± 1 — check exact recovery.

7.2 Criterion Benchmarks (`benches/core_bench.rs`)

One benchmark per major operation at representative sizes:

Benchmark	Input	Target
<code>bench_surface_512</code>	$z=f(x,y)$, 512×512	< 30 ms
<code>bench_implicit_128</code>	$F=0$, 128^3	< 120 ms
<code>bench_ode_200</code>	200 IVPs $\times$ 1000 steps	< 60 ms
<code>bench_batch_7step</code>	7-step Fourier scene	< 200 ms total
<code>bench_riemann_1000</code>	1000 rectangles	< 10 ms
<code>bench_vector_32</code>	32^3 arrows	< 40 ms
<code>bench_complex_1024</code>	1024×1024 coloring	< 100 ms

Criterion is configured to run with 100 samples and report p50/p95/p99. A CI step runs `cargo bench` and fails if any benchmark exceeds its target by more than 10%.

7.3 Property-Based Tests

Use `proptest` (similar to Hypothesis) for the expression evaluator and mesh generator:

- For any valid AST and domain spec, the evaluator must not panic.
- For any finite scalar field, the mesh generator must produce a valid index buffer (all indices within bounds, no degenerate triangles with duplicate vertex indices).
- For any valid matrix, the eigendecomposition must satisfy $M \cdot v = \lambda \cdot v$ within floating-point tolerance.

8. Security Constraints

The Rust core enforces these constraints unconditionally, independent of what the Python layer checks:

- No `unsafe` outside audited SIMD blocks. All `unsafe` usage must be in a dedicated `simd.rs` file with a comment explaining the invariants. No `unsafe` for general convenience.
- No `unwrap()` or `expect()` on the call path from Python. All `Option` and `Result` values must be explicitly handled. Use `?` operator or explicit match.

- **No recursion depth beyond 64 levels.** The AST walker must carry a depth counter and return `EvalError` if exceeded. This prevents stack overflow on pathologically nested expressions.
 - **NaN and Inf handling:** the evaluator returns `EvalError` if the result for any grid point is NaN or Inf, unless the caller specifies `allow_non_finite: true` (for implicit surface evaluation where sign changes through Inf are valid).
 - **Domain bounds enforced:** any x, y, z, t value is clamped to the domain before evaluation. The evaluator never evaluates outside the requested domain.
-

9. Build and Integration Notes

9.1 Cargo.toml Configuration

Required dependencies and their roles:

Crate	Purpose
<code>rayon</code>	Data-parallel iterators for all batch work
<code>nalgebra</code>	Matrix/vector types, eigendecomposition primitives
<code>ndarray</code>	N-dimensional array storage for scalar fields
<code>libm</code>	Portable math functions (sin, cos, etc.) for cross-platform SIMD safety
<code>pyo3</code> (feature: <code>extension-module</code>)	Python extension module scaffold
<code>pyo3-numpy</code>	Zero-copy NumPy array sharing
<code>maturin</code>	Build system for producing the <code>.so</code> / <code>.pyd</code> file
<code>serde</code> + <code>serde_json</code>	AST and DomainSpec deserialization from Python
<code>hashbrown</code>	Fast hash maps for within-call deduplication
<code>thiserror</code>	Ergonomic error type definition
<code>criterion</code> (dev)	Benchmarking
<code>proptest</code> (dev)	Property-based testing

Compile flags for release builds:

- `opt-level = 3`
- `lto = "thin"` (link-time optimization for cross-module inlining)
- `codegen-units = 1` (maximize optimization opportunities)
- `-C target-cpu=native` (enable SIMD for the deployment host)

9.2 Maturin Build

Maturin compiles the crate and produces a Python-importable `.so` file named `mathviz_core`. The file is baked into the Docker image at build time. The Python Visualization Service imports it as:

```
python
import mathviz_core
```

No Rust toolchain is required in the runtime container.

9.3 Feature Flags

Define the following optional Cargo features:

- `simd` — enables explicit SIMD intrinsics for complex arithmetic (default: on for release builds)
- `grpc-fallback` — compiles a gRPC server wrapper around the batch evaluator for multi-container deployments (default: off)
- `dev-logging` — enables structured logging of per-call timing for development profiling (default: off)

10. Extension Points

When adding a new concept type (e.g., a new mathematical structure):

1. Add the computation logic as a new function in the most relevant existing module, or create a new module following the same pattern.
2. Add a new `#[pyfunction]` in `lib.rs` or add the new concept as a variant to an existing dispatch function.
3. Add accuracy tests and a Criterion benchmark.
4. Update the Python Concept Handler to call the new function and wrap its output in the standard `GeometryBuffer` dict shape.

No changes to the PyO3 module registration pattern, the `BatchRequest` type, or the existing modules are required for additive extensions.